Компилятор ругается на opencl код.

```
#define TILE_SIZE 16
#define TS 16
#define WPT 4
#define TILE_M (TS * WPT) // 64
#define TILE_N (TS * WPT) // 64
#define TILE_K 16

//
================================================================================
=====
// 1. ПРЯМОЙ ПРОХОД GEMM: XW_out = X * W + bias (4x4 Register Tiling)
//
================================================================================
=====
kernel void lstm_gemm_xw(global const float *X, // [total_tokens, in_features]
                         global const float *W, // [in_features, total_gates]
                         global const float *bias, // [total_gates]
                         global float *XW_out, // [total_tokens, total_gates]
                         const int total_tokens, const int in_features,
                         const int total_gates) {
  int lx = get_local_id(0);
  int ly = get_local_id(1);
  int group_col = get_group_id(0) * TILE_N;
  int group_row = get_group_id(1) * TILE_M;

  __local float Xsub[TILE_M][TILE_K + 1];
  __local float Wsub[TILE_K][TILE_N + 1];

  float4 acc0 = (float4)(0.0f);
  float4 acc1 = (float4)(0.0f);
  float4 acc2 = (float4)(0.0f);
  float4 acc3 = (float4)(0.0f);

  int num_tiles = (in_features + TILE_K - 1) / TILE_K;

  for (int t = 0; t < num_tiles; t++) {
    int k_base = t * TILE_K;

    int x_col = k_base + lx;
    bool valid_x_col = (x_col < in_features);
#pragma unroll
    for (int w = 0; w < WPT; w++) {
      int local_r = ly + w * TS;
      int global_r = group_row + local_r;
      bool valid = (global_r < total_tokens) && valid_x_col;
      int safe_idx = select(0, global_r * in_features + x_col, valid);
      Xsub[local_r][lx] = select(0.0f, X[safe_idx], valid);
    }

    int w_row = k_base + ly;
    bool valid_w_row = (w_row < in_features);
```

```
#pragma unroll
    for (int w = 0; w < WPT; w++) {
      int local_c = lx + w * TS;
      int global_c = group_col + local_c;
      bool valid = valid_w_row && (global_c < total_gates);
      int safe_idx = select(0, w_row * total_gates + global_c, valid);
      Wsub[ly][local_c] = select(0.0f, W[safe_idx], valid);
    }

    barrier(CLK_LOCAL_MEM_FENCE);

#pragma unroll
    for (int k = 0; k < TILE_K; k++) {
      float4 x_vec = (float4)(Xsub[ly * WPT + 0][k], Xsub[ly * WPT + 1][k],
                              Xsub[ly * WPT + 2][k], Xsub[ly * WPT + 3][k]);
      float4 w_vec = vload4(0, &Wsub[k][lx * WPT]);

      acc0 = fma((float4)(x_vec.x), w_vec, acc0);
      acc1 = fma((float4)(x_vec.y), w_vec, acc1);
      acc2 = fma((float4)(x_vec.z), w_vec, acc2);
      acc3 = fma((float4)(x_vec.w), w_vec, acc3);
    }

    barrier(CLK_LOCAL_MEM_FENCE);
  }

  int out_col_base = group_col + lx * WPT;
  int out_row_base = group_row + ly * WPT;

  float4 b_vec = (float4)(select(0.0f,
                                 bias[select(0, out_col_base + 0,
                                             out_col_base + 0 < total_gates)],
                                 out_col_base + 0 < total_gates),
                          select(0.0f,
                                 bias[select(0, out_col_base + 1,
                                             out_col_base + 1 < total_gates)],
                                 out_col_base + 1 < total_gates),
                          select(0.0f,
                                 bias[select(0, out_col_base + 2,
                                             out_col_base + 2 < total_gates)],
                                 out_col_base + 2 < total_gates),
                          select(0.0f,
                                 bias[select(0, out_col_base + 3,
                                             out_col_base + 3 < total_gates)],
                                 out_col_base + 3 < total_gates));

  float4 rows =(float4)(acc0 + b_vec, acc1 + b_vec, acc2 + b_vec, acc3 + b_vec);

#pragma unroll
  for (int r = 0; r < 4; r++) {
    int gr = out_row_base + r;
    if (gr < total_tokens) {
```

```
      if (out_col_base + 3 < total_gates) {
        vstore4(rows[r], 0, XW_out + gr * total_gates + out_col_base);
      } else {
        float vals = {rows[r].x, rows[r].y, rows[r].z, rows[r].w};
        for (int c = 0; c < 4; c++) {
          if (out_col_base + c < total_gates) {
            XW_out[gr * total_gates + out_col_base + c] = vals[c];
          }
        }
      }
    }
  }
}


kernel void lstm_gemm_grad_x(
    global const float *all_grad_gates, // [total_tokens, total_gates]
    global const float *W,              // [in_features, total_gates]
    global float *grad_X,               // [total_tokens, in_features]
    const int total_tokens, const int in_features, const int total_gates) {
  int lx = get_local_id(0);
  int ly = get_local_id(1);
  int group_col = get_group_id(0) * TILE_N; // по оси in_features
  int group_row = get_group_id(1) * TILE_M; // по оси total_tokens

  __local float Gsub[TILE_M][TILE_K + 1];
  __local float WTsub[TILE_K][TILE_N + 1];

  float4 acc0 = (float4)(0.0f);
  float4 acc1 = (float4)(0.0f);
  float4 acc2 = (float4)(0.0f);
  float4 acc3 = (float4)(0.0f);

  int num_tiles = (total_gates + TILE_K - 1) / TILE_K;

  for (int t = 0; t < num_tiles; t++) {
    int k_base = t * TILE_K;

    // Загрузка Gsub[64 токена][16 гейтов]
    int g_col = k_base + lx;
    bool valid_g_col = (g_col < total_gates);
#pragma unroll
    for (int w = 0; w < WPT; w++) {
      int local_r = ly + w * TS;
      int global_r = group_row + local_r;
      bool valid = (global_r < total_tokens) && valid_g_col;
      int safe_idx = select(0, global_r * total_gates + g_col, valid);
      Gsub[local_r][lx] = select(0.0f, all_grad_gates[safe_idx], valid);
    }

    // Загрузка W^T[16 гейтов][64 признака] -> W[global_c * total_gates + w_k]
    int w_k = k_base + ly;
```

```
    bool valid_w_k = (w_k < total_gates);
#pragma unroll
    for (int w = 0; w < WPT; w++) {
      int local_c = lx + w * TS;
      int global_c = group_col + local_c;
      bool valid = valid_w_k && (global_c < in_features);
      int safe_idx = select(0, global_c * total_gates + w_k, valid);
      WTsub[ly][local_c] = select(0.0f, W[safe_idx], valid);
    }

    barrier(CLK_LOCAL_MEM_FENCE);

#pragma unroll
    for (int k = 0; k < TILE_K; k++) {
      float4 g_vec = (float4)(Gsub[ly * WPT + 0][k], Gsub[ly * WPT + 1][k],
                              Gsub[ly * WPT + 2][k], Gsub[ly * WPT + 3][k]);
      float4 wt_vec = vload4(0, &WTsub[k][lx * WPT]);

      acc0 = fma((float4)(g_vec.x), wt_vec, acc0);
      acc1 = fma((float4)(g_vec.y), wt_vec, acc1);
      acc2 = fma((float4)(g_vec.z), wt_vec, acc2);
      acc3 = fma((float4)(g_vec.w), wt_vec, acc3);
    }

    barrier(CLK_LOCAL_MEM_FENCE);
  }

  int out_col_base = group_col + lx * WPT;
  int out_row_base = group_row + ly * WPT;
  float4 rows = {acc0, acc1, acc2, acc3};

#pragma unroll
  for (int r = 0; r < 4; r++) {
    int gr = out_row_base + r;
    if (gr < total_tokens) {
      if (out_col_base + 3 < in_features) {
        vstore4(rows[r], 0, grad_X + gr * in_features + out_col_base);
      } else {
        float vals = {rows[r].x, rows[r].y, rows[r].z, rows[r].w};
        for (int c = 0; c < 4; c++) {
          if (out_col_base + c < in_features) {
            grad_X[gr * in_features + out_col_base + c] = vals[c];
          }
        }
      }
    }
  }
}
//
================================================================================
=====
// ЯДРО 3: ТАЙЛОВЫЙ GEMM ДЛЯ grad_W, grad_U И grad_bias (БЕЗ АТОМИКОВ!)
```

```
//
================================================================================
=====
kernel void lstm_gemm_grad_weights(
    global const float *X,              // [total_tokens, in_features]
    global const float *H_out,          // [batch_size, seq_len, hidden_size]
    global const float *all_grad_gates, // [total_tokens, total_gates]
    global float *grad_W,               // [in_features, total_gates]
    global float *grad_U,               // [hidden_size, total_gates]
    global float *grad_bias,            // [total_gates]
    const int batch_size, const int seq_len, const int in_features,
    const int hidden_size) {
  int lx = get_local_id(0);
  int ly = get_local_id(1);
  int group_col = get_group_id(0) * TILE_N; // по оси total_gates
  int group_row = get_group_id(1) * TILE_M; // по оси in_features / hidden_size

  int total_tokens = batch_size * seq_len;
  int total_gates = 4 * hidden_size;
  int max_rows = max(in_features, hidden_size);

  if (group_col >= total_gates || group_row >= max_rows)
    return;

  __local float LeftSub[TILE_M][TILE_K + 1]; // X^T или H_prev^T [64][16]
  __local float Gsub[TILE_K][TILE_N + 1];    // all_grad_gates [16][64]

  int num_tiles = (total_tokens + TILE_K - 1) / TILE_K;
  int out_col_base = group_col + lx * WPT;
  int out_row_base = group_row + ly * WPT;

  // ----------------------------------------------------------------------
  // ЧАСТЬ A: grad_W += X^T * all_grad_gates и векторный grad_bias
  // ----------------------------------------------------------------------
  if (group_row < in_features) {
    float4 acc0 = (float4)(0.0f);
    float4 acc1 = (float4)(0.0f);
    float4 acc2 = (float4)(0.0f);
    float4 acc3 = (float4)(0.0f);
    float4 acc_bias = (float4)(0.0f);

    bool compute_bias = (get_group_id(1) == 0 && ly == 0);

    for (int t = 0; t < num_tiles; t++) {
      int k_base = t * TILE_K;

      // Загружаем X^T[64 признака][16 токенов]
      int tok_x = k_base + lx;
      bool valid_tok_x = (tok_x < total_tokens);
#pragma unroll
      for (int w = 0; w < WPT; w++) {
        // Используйте код с осторожностью.
```

```
      int local_r = ly + w * TS;
      int global_r = group_row + local_r;
      bool valid = (global_r < in_features) && valid_tok_x;
      int safe_idx = select(0, tok_x * in_features + global_r, valid);
      LeftSub[local_r][lx] = select(0.0f, X[safe_idx], valid);
    }
    // Загружаем G[16 токенов][64 гейта]
    int tok_y = k_base + ly;
    bool valid_tok_y = (tok_y < total_tokens);
#pragma unroll
    for (int w = 0; w < WPT; w++) {
      int local_c = lx + w * TS;
      int global_c = group_col + local_c;
      bool valid = valid_tok_y && (global_c < total_gates);
      int safe_idx = select(0, tok_y * total_gates + global_c, valid);
      Gsub[ly][local_c] = select(0.0f, all_grad_gates[safe_idx], valid);
    }
    barrier(CLK_LOCAL_MEM_FENCE);
#pragma unroll
    for (int k = 0; k < TILE_K; k++) {
      float4 x_vec =
          (float4)(LeftSub[ly * WPT + 0][k], LeftSub[ly * WPT + 1][k],
                   LeftSub[ly * WPT + 2][k], LeftSub[ly * WPT + 3][k]);
      float4 g_vec = vload4(0, &Gsub[k][lx * WPT]);
      acc0 = fma((float4)(x_vec.x), g_vec, acc0);
      acc1 = fma((float4)(x_vec.y), g_vec, acc1);
      acc2 = fma((float4)(x_vec.z), g_vec, acc2);
      acc3 = fma((float4)(x_vec.w), g_vec, acc3);
      if (compute_bias) {
        acc_bias += g_vec;
      }
    }
    barrier(CLK_LOCAL_MEM_FENCE);
  }
  // Накопление grad_W (оставляем под if для защиты от гонки)
  float4 rows = {acc0, acc1, acc2, acc3};
#pragma unroll
  for (int r = 0; r < 4; r++) {
    int gr = out_row_base + r;
    if (gr < in_features) {
      if (out_col_base + 3 < total_gates) {
        int offset = gr * total_gates + out_col_base;
        vstore4(vload4(0, grad_W + offset) + rows[r], 0, grad_W + offset);
      } else {
        float vals = {rows[r].x, rows[r].y, rows[r].z, rows[r].w};
        for (int c = 0; c < 4; c++) {
          if (out_col_base + c < total_gates) {
            grad_W[gr * total_gates + out_col_base + c] += vals[c];
          }
        }
      }
    }
```

```
    }
    // Накопление grad_bias
    if (compute_bias) {
      if (out_col_base + 3 < total_gates) {
        vstore4(vload4(0, grad_bias + out_col_base) + acc_bias, 0,
                grad_bias + out_col_base);
      } else {
        float b_vals = {acc_bias.x, acc_bias.y, acc_bias.z, acc_bias.w};
        for (int c = 0; c < 4; c++) {
          if (out_col_base + c < total_gates) {
            grad_bias[out_col_base + c] += b_vals[c];
          }
        }
      }
    }
  }
  // ---------------------------------------------------------------------
  // ЧАСТЬ Б: grad_U += H_prev^T * all_grad_gates
  // ---------------------------------------------------------------------
  if (group_row < hidden_size) {
    float4 acc0 = (float4)(0.0f);
    float4 acc1 = (float4)(0.0f);
    float4 acc2 = (float4)(0.0f);
    float4 acc3 = (float4)(0.0f);
    for (int t = 0; t < num_tiles; t++) {
      int k_base = t * TILE_K;
      // Загружаем H_prev^T[64 нейрона][16 токенов] со склеенным условием (tok_x
      // % seq_len) > 0
      int tok_x = k_base + lx;
      bool valid_tok_h = (tok_x < total_tokens) && ((tok_x % seq_len) > 0);
#pragma unroll
      for (int w = 0; w < WPT; w++) {
        int local_r = ly + w * TS;
        int global_r = group_row + local_r;
        bool valid = (global_r < hidden_size) && valid_tok_h;
        int safe_idx = select(0, (tok_x - 1) * hidden_size + global_r, valid);
        LeftSub[local_r][lx] = select(0.0f, H_out[safe_idx], valid);
      }
      // Загружаем G[16 токенов][64 гейта]
      int tok_y = k_base + ly;
      bool valid_tok_y = (tok_y < total_tokens);
#pragma unroll
      for (int w = 0; w < WPT; w++) {
        int local_c = lx + w * TS;
        int global_c = group_col + local_c;
        bool valid = valid_tok_y && (global_c < total_gates);
        int safe_idx = select(0, tok_y * total_gates + global_c, valid);
        Gsub[ly][local_c] = select(0.0f, all_grad_gates[safe_idx], valid);
      }
      barrier(CLK_LOCAL_MEM_FENCE);
#pragma unroll
      for (int k = 0; k < TILE_K; k++) {
```

```
      float4 h_vec =
          (float4)(LeftSub[ly * WPT + 0][k], LeftSub[ly * WPT + 1][k],
                   LeftSub[ly * WPT + 2][k], LeftSub[ly * WPT + 3][k]);
      float4 g_vec = vload4(0, &Gsub[k][lx * WPT]);
      acc0 = fma((float4)(h_vec.x), g_vec, acc0);
      acc1 = fma((float4)(h_vec.y), g_vec, acc1);
      acc2 = fma((float4)(h_vec.z), g_vec, acc2);
      acc3 = fma((float4)(h_vec.w), g_vec, acc3);
    }
    barrier(CLK_LOCAL_MEM_FENCE);
  }
  float4 rows = {acc0, acc1, acc2, acc3};
#pragma unroll
  for (int r = 0; r < 4; r++) {
    int gr = out_row_base + r;
    if (gr < hidden_size) {
      if (out_col_base + 3 < total_gates) {
        int offset = gr * total_gates + out_col_base;
        vstore4(vload4(0, grad_U + offset) + rows[r], 0, grad_U + offset);
      } else {
        float vals = {rows[r].x, rows[r].y, rows[r].z, rows[r].w};
        for (int c = 0; c < 4; c++) {
          if (out_col_base + c < total_gates) {
            grad_U[gr * total_gates + out_col_base + c] += vals[c];
          }
        }
      }
    }
  }
}
```